

Register Transfer and Microoperations

Objective

This chapter introduces a register transfer language and shows how it is used to express microoperations in symbolic form. Symbols are defined for arithmetic, logic, and shift microoperations. A composite arithmetic, logic shift unit is developed to show the hardware design of the most common microoperations.

OUTLINE

- 2.1 Register Transfer Language
- 2.2 Register Transfer
- 2.3 Bus and Memory Transfers
- 2.4 Arithmetic Microoperations
- 2.5 Logic Microoperations
- 2.6 Shift Microoperations
- 2.7 Arithmetic Logic shift Unit

2.1 INTRODUCTION

Computers are classified according to **functionality**, **physical size** and **purpose**. Functionality, Computers could be **analog**, **digital** or **hybrid**. Digital computers process data that is in discrete form whereas analog computers process data that is continuous in nature. Hybrid computers on the other hand can process data that is both discrete and continuous.

In digital computers, the user input is first converted and transmitted as electrical pulses that can be represented by two unique states ON and OFF. The ON state may be represented by a "1" and the off state by a "0". The sequence of ON'S and OFF'S forms the electrical signals that the computer can understand. A digital signal rises suddenly to a peak voltage of +1 for some time then suddenly drops -1 level on the other hand an analog signal rises to +1 and then drops to -1 in a continuous version.

2.2 REGISTER TRANSFER LANGUAGE

Digital systems are composed of modules that are constructed from digital components, such as registers, decoders, arithmetic elements, and control logic. The modules are interconnected with common data and control paths to form a digital computer system. The operations executed on data stored in registers are called *microoperations*. A microoperation is an elementary operation performed on the information stored in one or more registers. Examples are shift, count, clear, and load. Some of the digital components from before are registers that implement microoperations. The internal hardware organization of a digital computer is best defined by specifying:

- ▲ The set of registers it contains and their functions
- ▲ The sequence of microoperations performed on the binary information stored
- ▲ The control that initiates the sequence of microoperations

It uses symbols, rather than words, to specify the sequence of microoperations. The symbolic notation used is called a *register transfer language*. A programming language is a procedure for writing symbols to specify a given computational process. RTL defines symbols for various types of microoperations and describe associated hardware that can implement the microoperations

2.3 REGISTER TRANSFER

Designate computer registers by capital letters to denote its function. The register that holds an address for the memory unit is called MAR. The program counter register is called PC

IR is the instruction register and R_1 is a processor register. The individual flip-flops in an n -bit register are numbered in sequence from 0 to $n-1$ Refer to Figure 4.1 for the different representations of a register

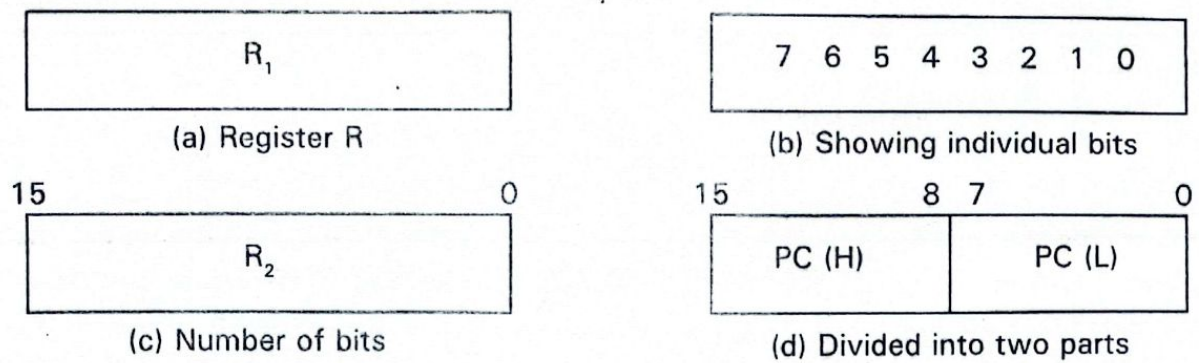
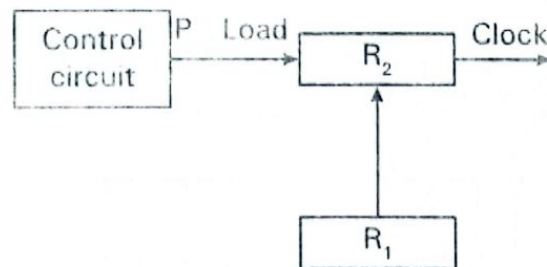


Figure 2.1: Different representations of a register

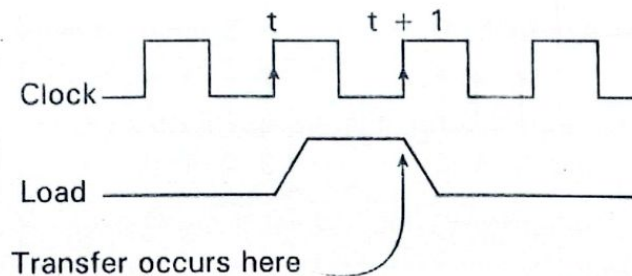
- Designate information transfer from one register to another by $R_2 \leftarrow R_1$
- This statement implies that the hardware is available
 - ▲ The outputs of the source must have a path to the inputs of the destination
 - ▲ The destination register has a parallel load capability
- If the transfer is to occur only under a predetermined control condition, designate it by
 or, If ($P = 1$) then ($R_2 \rightarrow R_1$)
 $P : R_2 \leftarrow R_1$

where P is a control function that can be either 0 or 1

- Every statement written in register transfer notation implies the presence of the required hardware construction



(a) Block diagram



(b) Timing diagram

Figure 2.2: Transfer from R_1 to R_2 when $P = 1$

- It is assumed that all transfers occur during a clock edge transition
- All microoperations written on a single line are to be executed at the same time
 $T: R_2 \leftarrow R_1, R_1 \leftarrow R_2$

Symbol	Description	Examples
Letters and numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of register	R2(0 - 7), R2(L)
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two microoperations	$R2 \leftarrow R1, R1 \leftarrow R2$

Table 2.1 : Basic Symbols for Register Transfer

2.4 BUS AND MEMORY TRANSFERS

Rather than connecting wires between all registers, a common bus is used. A bus structure consists of a set of common lines, one for each bit of a register where control signals determine which register is selected by the bus during each transfer. Multiplexers can be used to construct a common bus and it selects the source register whose binary information is then placed on the bus. The select lines are connected to the selection inputs of the multiplexers and choose the bits of one register

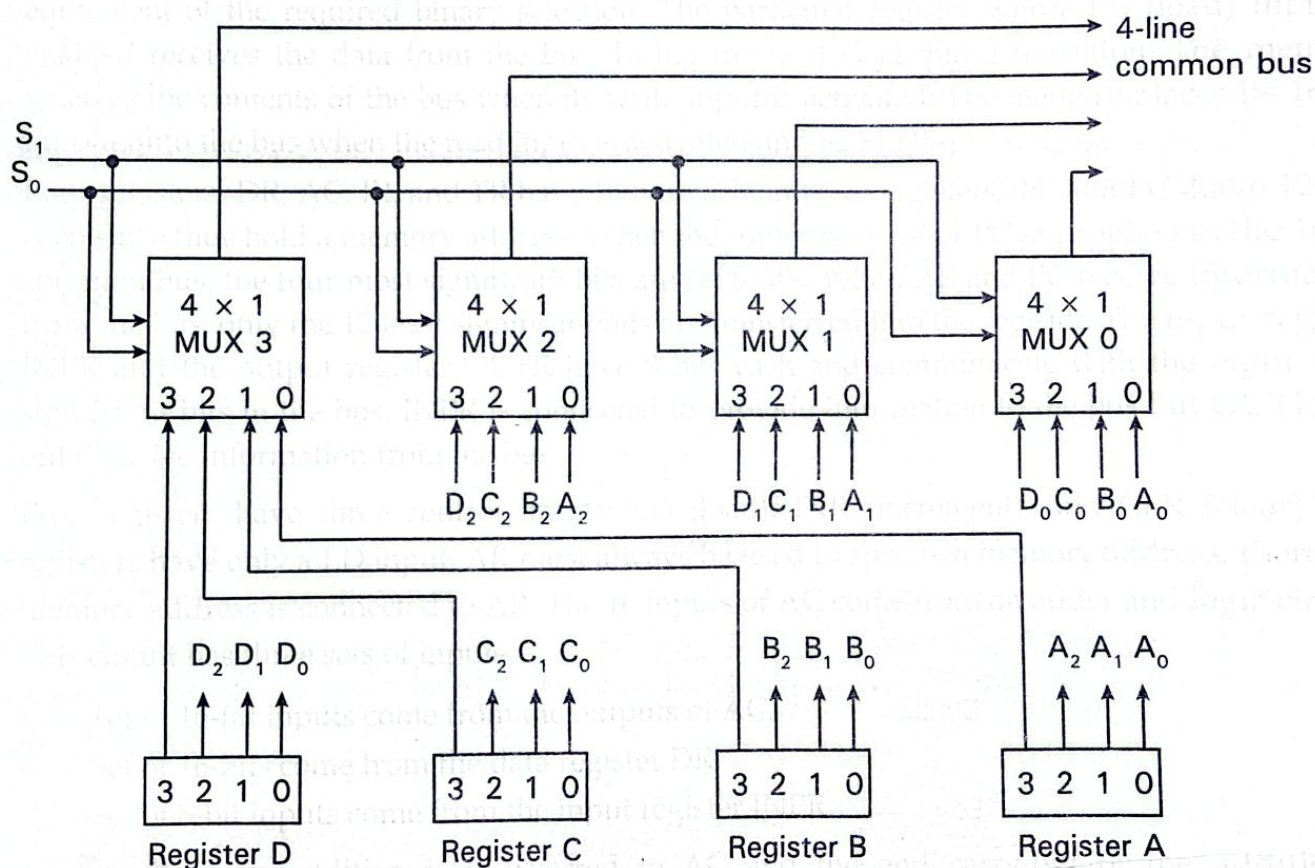


Figure 2.3: Bus system for four registers

S_1	S_0	Register Selected
0	0	A
0	1	B
1	0	C
1	1	D

Table 2.2 : Function table for bus of Fig. 2.3

In general, a bus system will multiplex k registers of n bits each to produce an n -line common bus. This requires n multiplexers – one for each bit. The size of each multiplexer must be $k \times 1$ and the number of select lines required is $\log k$. To transfer information from the bus to a register, the bus lines are connected to the inputs of all destination registers and the corresponding load control line must be activated rather than listing each step as $BUS \leftarrow C$, $R_1 \leftarrow BUS$, use $R_1 \leftarrow C$, since the bus is implied. Instead of using multiplexers, *three-state gates* can be used to construct the bus system. A three-state gate is a digital circuit that exhibits three states where two of the states are signals equivalent to logic 1 and 0 and the third state is a *high-impedance* state – this behaves like an open circuit, which means the output is disconnected and does not have a logic significance

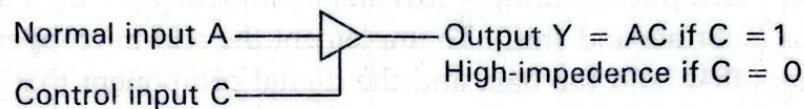


Figure 2.4: Graphical symbols for three-state buffer

The three-state buffer gate has a normal input and a control input which determines the output state: With control 1, the output equals the normal input; with control 0, the gate goes to a high-impedance state; this enables a large number of three-state gate outputs to be connected with wires to form a common bus line without endangering loading effects

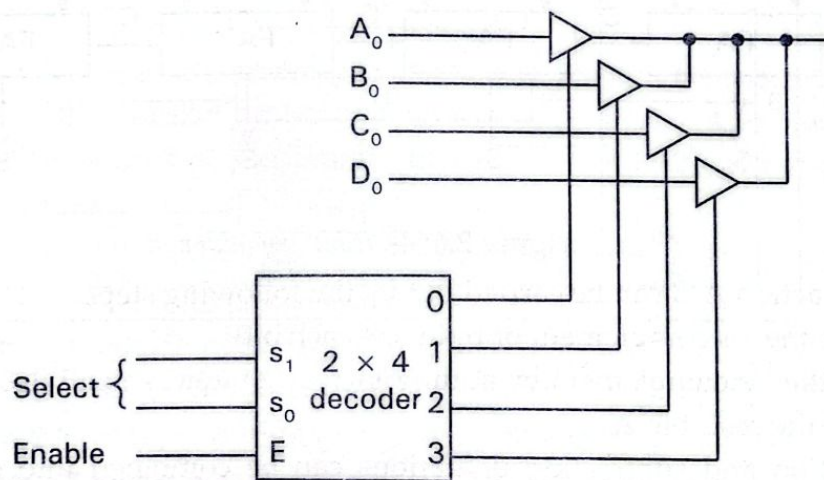


Figure 2.5: Bus line with three state-buffers

2.5 ARITHMETIC MICROOPERATIONS

- There are four categories of the most common microoperations:
 - ▲ **Register transfer:** transfer binary information from one register to another
 - ▲ **Arithmetic:** perform arithmetic operations on numeric data stored in registers
 - ▲ **Logic:** perform bit manipulation operations on non-numeric data stored in registers
 - ▲ **Shift:** perform shift operations on data stored in registers

The basic arithmetic microoperations are addition, subtraction, increment, decrement, and shift. An example of addition: $R_3 \leftarrow R_1 + R_2$ and subtraction is most often implemented through complementation and addition. Example of subtraction: $R_3 \leftarrow R_1 + \bar{R}_2 + 1$

Symbolic designation	Description
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow \overline{R2}$	Complement of the contents of R2 (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement the content of R2 (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	R1 plus the 2's complement of R2 (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by one
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by one

Table 2.3 : Arithmetic Microoperations

Multiply and divide are not included as microoperations. A microoperation is one that can be executed by one clock pulse. Multiply (divide) is implemented by a sequence of add and shift microoperations (subtract and shift). To implement the add microoperation with hardware, we need the registers that hold the data and the digital component that performs the addition. A full-adder adds two bits and a previous carry.

A binary adder is a digital circuit that generates the arithmetic sum of two binary numbers of any length where a binary added is constructed with full-adder circuits connected in cascade. An n -bit binary adder requires n full-adders.

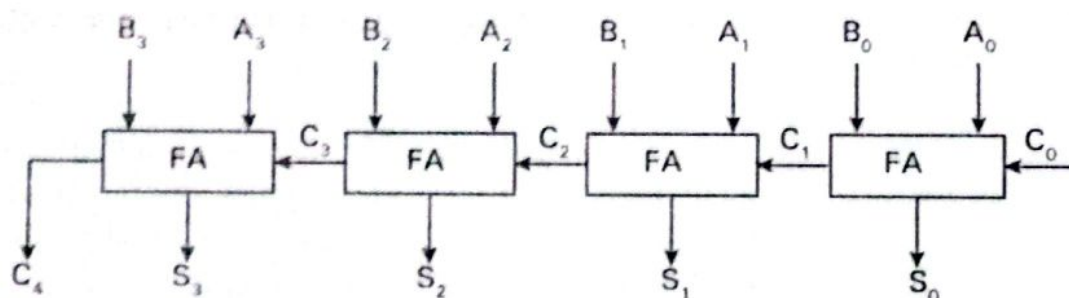


Figure 2.6: 4-bit binary adder

- The subtraction $A-B$ can be carried out by the following steps
 - Take the 1's complement of B (invert each bit)
 - Get the 2's complement by adding 1
 - Add the result to A
- The addition and subtraction operations can be combined into one common circuit by including an XOR gate with each full-adder

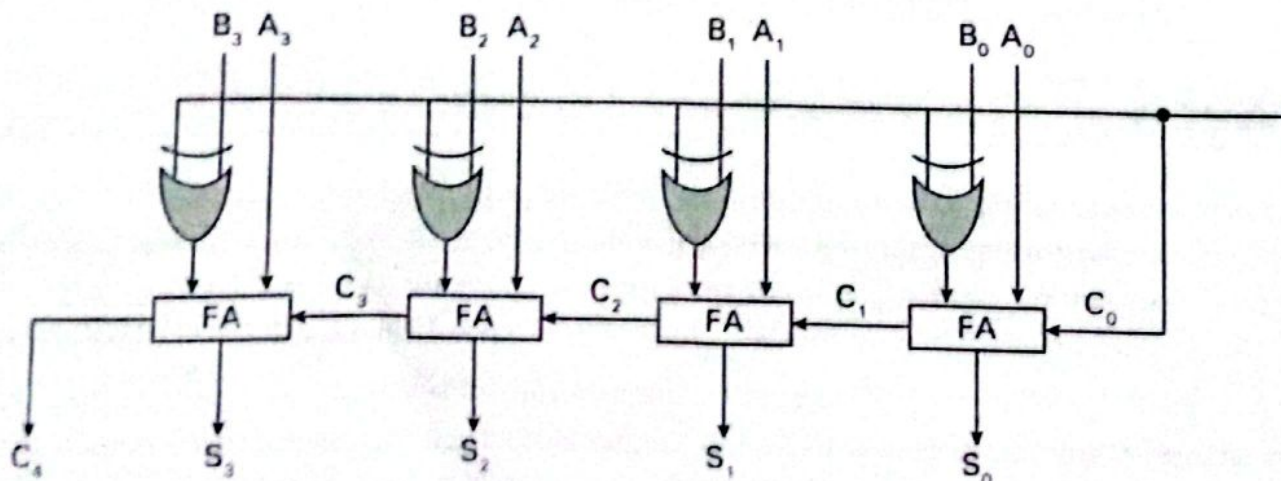


Figure 2.7: 4-bit adder-subtractor

The increment microoperation adds one to a number in a register. This can be implemented by using a binary counter - every time the count enable is active, the count is incremented by one. If the increment is to be performed independent of a particular register, then use half-adders connected in cascade. An n -bit binary incrementer requires n half-adders.

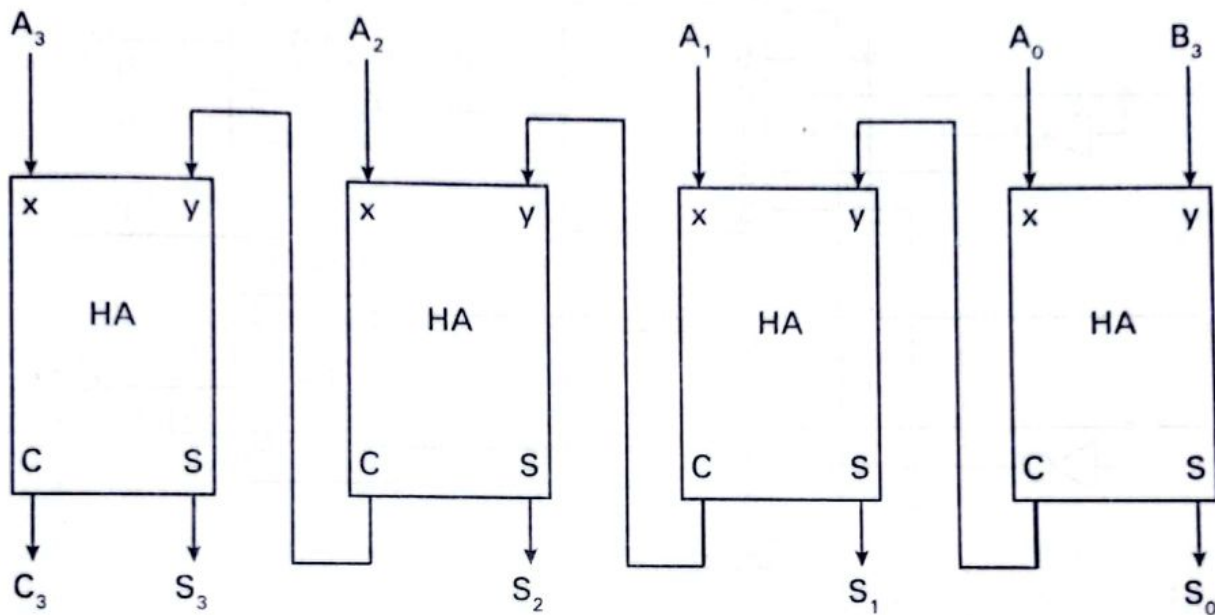


Figure 2.8: 4-bit binary incrementer

Arithmetic Circuit

Each of the arithmetic microoperations can be implemented in one composite arithmetic circuit. The basic component is the parallel adder and multiplexers are used to choose between the different operations. The output of the binary adder is calculated from the following sum:

$$D = A + Y + C_{in}$$

- **Addition:** when $S_1S_0 = 00$, $Y = B$. If $C_{in} = 0$, the output $d = A + B$. If $C_{in} = 1$, output $D = A + B + 1$.
- **Subtraction:** when $S_1S_0 = 01$, $Y = \bar{B}$. If $C_{in} = 1$, then $D = A + \bar{B} + 1 = A - B$. When $C_{in} = 0$ then $D = A + \bar{B} = A - B - 1$.
- **Increment:** When $S_1S_0 = 10$, $Y = 0$ (transfer), the output $d = A + 0 + C_{in}$ (increment). This gives $D = A$ (transfer) when $C_{in} = 0$ and $D = A + 1$ (increment) when $C_{in} = 1$.
- **Decrement:** When $S_1S_0 = 11$, all is insulated into Y inputs hence $D = A - 1$ when $C_{in} = 0$. This is because a number with all 1's is equal to the 2's complement of 1 (the 2's complement of binary 0001 is 1111). Adding a number a to 2's complement of 1 produces $F = A + 2's \text{ complement of } 1 = A - 1$. when $C_{in} = 1$, then $D = A - 1 + 1 = A$ which causes a direct transfer from input A to output D .

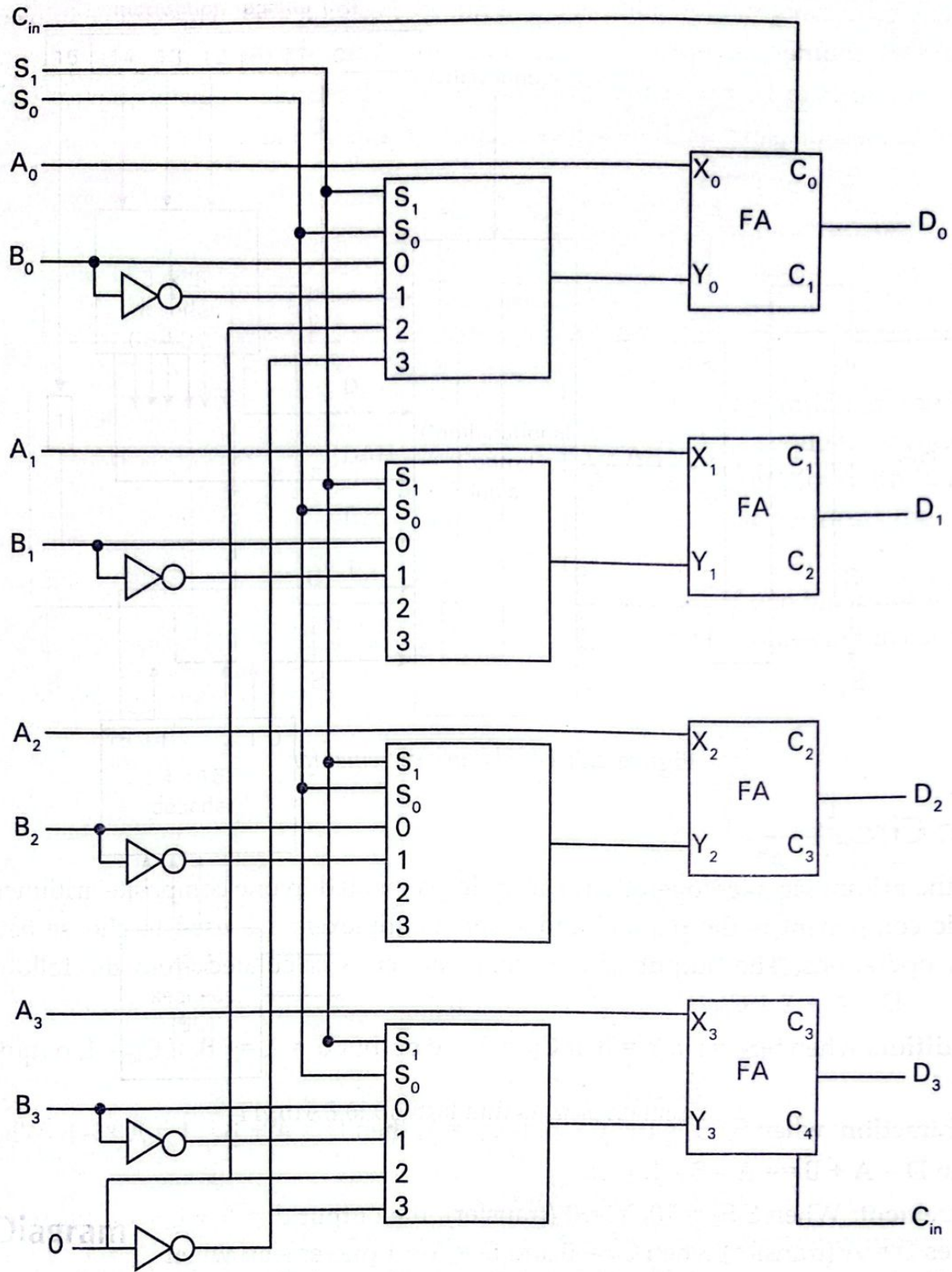


Figure 2.9: 4-bit arithmetic circuit

Select		C _{in}	Input Y	Output		Microoperation
S ₁	S ₀			D = A + Y + C _{in}		
0	0	0	B	D = A + B		Add
0	0	1	B	D = A + B + 1		Add with carry
0	1	0	B	D = A + $\bar{B}$		Subtract with borrow
0	1	1	$\bar{B}$	D = A + $\bar{B}$ + 1		Subtract
1	0	0	0	D = A		Transfer A
1	0	1	0	D = A + 1		Increment A
1	1	0	1	D = A - 1		Decrement A
1	1	1	1	D = A		Transfer A

Table 2.4: Arithmetic circuit function table

2.6 LOGIC MICROOPERATIONS

Logic operations specify binary operations for strings of bits stored in registers and treat each bit separately. Example: the XOR of R1 and R2 is symbolized by

$$P: R_1 \leftarrow R_1 \oplus R_2$$

Example: $R_1 = 1010$ and $R_2 = 1100$

1010 Content of R_1

1100 Content of R_2

0110 Content of R_1 after $P = 1$

▪ Symbols used for logical microoperations:

▲ **OR:** $\vee$

▲ **AND:** $\wedge$

▲ **XOR:** $\oplus$

The $+$ sign has two different meanings: logical OR and summation, when $+$ is in a microoperation, then summation and when $+$ is in a control function, then OR. Example:

$$P + Q: R_1 \leftarrow R_2 + R_3, R_4 \leftarrow R_5 \vee R_6$$

There are 16 different logic operations that can be performed with two binary variables

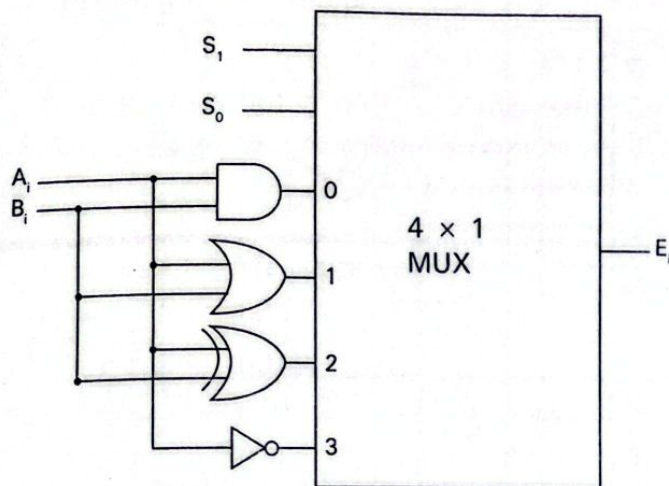
x	y	F ₀	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇	F ₈	F ₉	F ₁₀	F ₁₁	F ₁₂	F ₁₃	F ₁₄	F ₁₅
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Table 2-5: Truth tables for 16 functions of two variables

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \bar{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \bar{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \bar{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \bar{B}$	
$F_{12} = x'$	$F \leftarrow \bar{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \bar{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Table 2-6: Sixteen logic microoperations

The hardware implementation of logic microoperations requires that logic gates be inserted for each bit or pair of bits in the registers. All 16 microoperations can be derived from using four logic gates



(a) Logic diagram

S ₁	S ₀	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

Figure: 2-10: One stage of logic circuit

- Logic microoperations can be used to change bit values, delete a group of bits, or insert new bit values into a register
- The *selective-set* operation sets to 1 the bits in A where there are corresponding 1's in B
 - $\wedge$ 1010 A before
 - $\wedge$ 1100 B (logic operand) 1110 A after
 - $\wedge$ $A \leftarrow A \vee B$
- The *selective-complement* operation complements bits in A where there are corresponding 1's in B
 - $\wedge$ 1010 A before
 - $\wedge$ 1100 B (logic operand) 0110 A after
 - $\wedge$ $A \leftarrow A \oplus B$
- The *selective-clear* operation clears to 0 the bits in A only where there are corresponding 1's in B
 - $\wedge$ 1010 A before
 - $\wedge$ 1100 B (logic operand) 0010 A after
 - $\wedge$ $A \leftarrow A \wedge \bar{B}$
- The *mask* operation is similar to the selective-clear operation, except that the bits of A are cleared only where there are corresponding 0's in B
 - $\wedge$ 1010 A before
 - $\wedge$ 1100 B (logic operand) 1000 A after
 - $\wedge$ $A \leftarrow A \wedge B$
- The *insert* operation inserts a new value into a group of bits
- This is done by first masking the bits to be replaced and then Oring them with the bits to be inserted

0110 1010	A before
<u>0000 1111</u>	B (mask)
0000 1010	A after masking
0000 1010	A before
<u>1001 0000</u>	B (insert)
1001 1010	A after insertion

- The *clear* operation compares the bits in A and B and produces an all 0's result if the two number are equal

$\wedge$ 1010 A
 $\wedge$ 1010 B
 $\wedge$ 0000 $A \leftarrow A \oplus B$

2.7 SHIFT MICROOPERATIONS

Shift microoperations are used for serial transfer of data. They are also used in conjunction with arithmetic, logic, and other data-processing operations and there are three types of shifts: logical, circular, and arithmetic. A *logical shift* is one that transfers 0 through the serial input. The symbols *shl* and *shr* are for logical shift-left and shift-right by one position $R_1 \leftarrow \text{shl } R_1$. The *circular shift* (aka rotate) circulates the bits of the register around the two ends without loss of information. The symbols *cil* and *cir* are for circular shift left and right.

Symbolic designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic shift-left R
$R \leftarrow \text{ashr } R$	Arithmetic shift-right R

Table2-7: Shift microoperations

The *arithmetic shift* shifts a signed binary number to the left or right, to the left is multiplying by 2, to the right is dividing by 2 and arithmetic shifts must leave the sign bit unchanged. A sign reversal occurs if the bit in R_{n-1} changes in value after the shift, this happens if the multiplication causes an overflow. An overflow flip-flop V_s can be used to detect the overflow $V_s = R_{n-1} \oplus R_{n-2}$

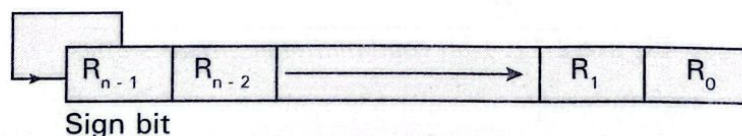
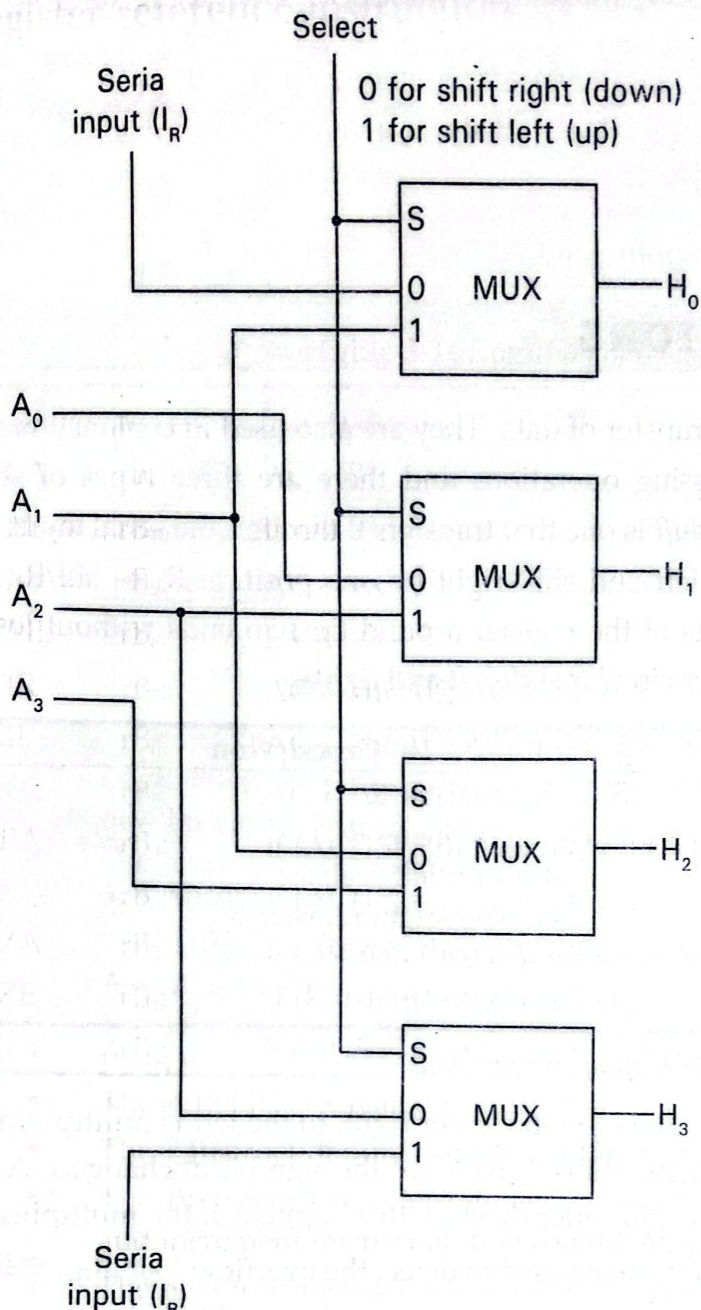


Figure 2-11: Arithmetic shift right

A bi-directional shift unit with parallel load could be used to implement this kind of shifting operation. Here, two clock pulses are necessary with this configuration: one to load the value and another to shift. In a processor unit with many registers it is more efficient to implement the shift operation with a combinational circuit. The content of a register to be shifted is first placed onto a common bus and the output is connected to the combinational shifter, the shifted number is then loaded back into the register. This can be constructed with multiplexers.

Function Table				
Select	Output			
S	H ₀	H ₁	H ₂	H ₃
0	I _R	A ₀	A ₁	A ₂
1	A ₁	A ₂	A ₃	I _L



Function Table				
Select	Output			
S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

Figure 2-12: 4-bit combinational circuit shifter

2.8 ARITHMETIC LOGIC SHIFT UNIT

The arithmetic logic unit (ALU) is a common operational unit connected to a number of storage registers. To perform a microoperation, the contents of specified registers are placed in the inputs of the ALU. The ALU performs an operation and the result is then transferred to a destination register. The ALU is a combinational circuit so that the entire register transfer operation from the source registers through the ALU and into the destination register can be performed during one clock pulse period.

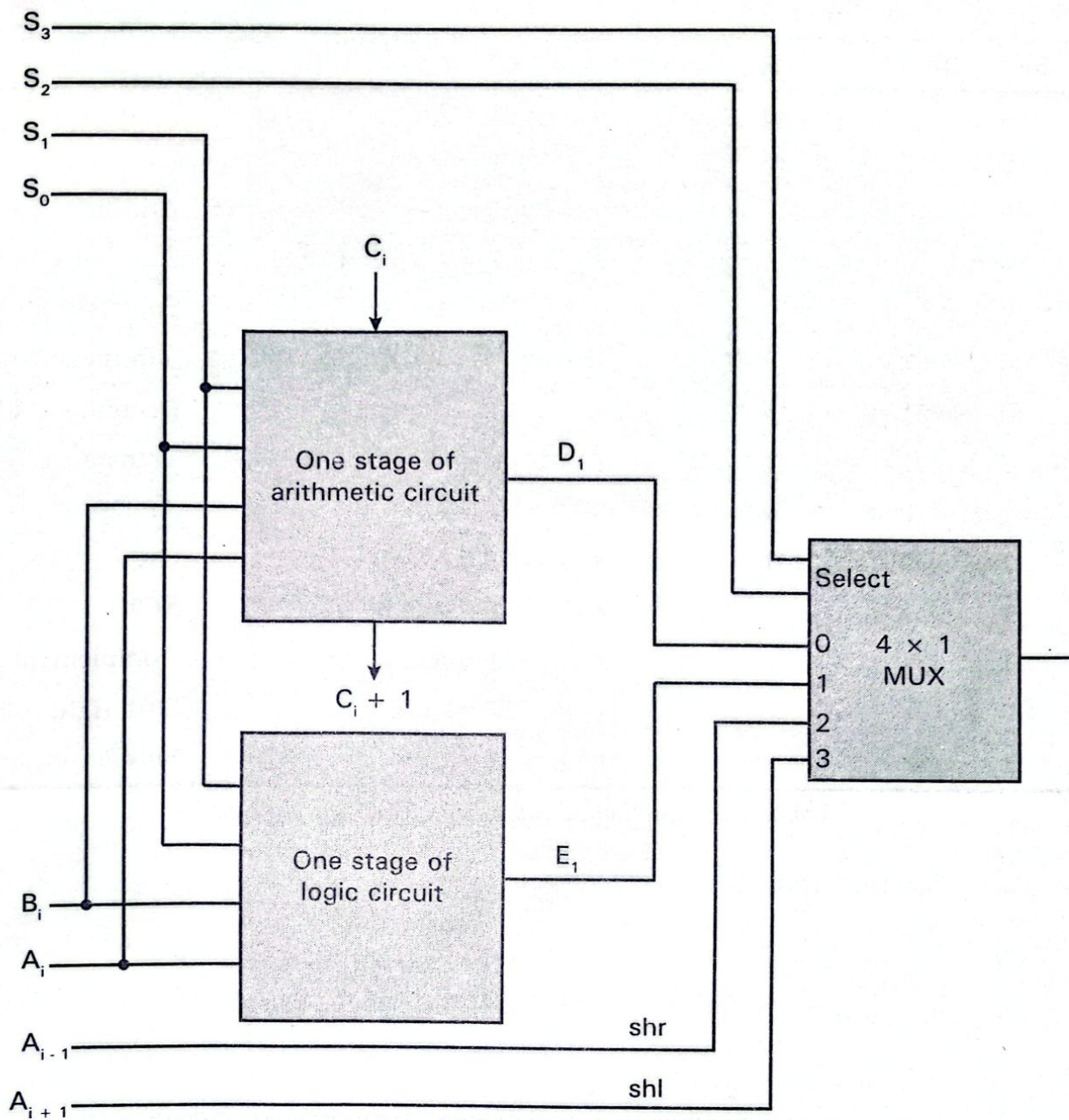


Figure 2-13; One stage of arithmetic logic-shift unit

The arithmetic, logic and shift circuits introduced in previous sections can be combined into one ALU with common selection variables. One stage of an arithmetic logic shift unit is shown in Fig. 2.3. The subscript designates a typical stage. Inputs $A=i$ and B_i are applied to both the arithmetic and logic units. A particular microoperation is selected with inputs S_1 and S_0 . A 4×2 multiplier at the output chooses between an arithmetic output E_i and a logic output in H_i . The data in multiplier are selected with inputs SS_3 and S_2 . The other two data inputs to the multiplier receive inputs A_{i-1} for the shift right operation and A_{i+1} for the shift left operation.

Operation Select					Operation	Function
S_3	S_2	S_1	S_0	C_{in}		
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \bar{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \bar{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	$\times$	$F = A \wedge B$	AND
0	1	0	1	$\times$	$F = A \vee B$	OR
0	1	1	0	$\times$	$F = A \oplus B$	XOR
0	1	1	1	$\times$	$F = \bar{A}$	Complement A
1	0	$\times$	$\times$	$\times$	$F = \text{shr } r$	Shift right A into F
1	1	$\times$	$\times$	$\times$	$F = \text{shl } A$	Shift left A into F

Table 2-8: Function table for arithmetic logic shift unit

